Technical Report - **Product Specification**

EasySpot

Course: IES - Introdução à Engenharia de Software

Date: Aveiro, 27/05/2026

Students: 127368: Claudino Martins
125895: Inês Lourenço
125102: Maria Mané
124833: Martim Gil

Project Abstract: EasySpot is a real-time parking analytics platform that shows live parking occupancy, EV charger status, and accessible-bay availability across urban sites.

Table of Contents:

1. Summary 4
2. Product concept 4
    - Concept 4
    - Inspiration 4
    - Motivation 4
    - Persona #1: The regular driver 5
    - Persona #2 : The EV (Electric Vehicle) Driver 5
    - Persona #3 : The Mobility-Impaired Driver 6
    - Persona #4 : The Parking Manager 6
    - Persona #5 : The Maintenance Technician 6
    - Scenario #2: Maria Silva (EV driver) 7
    - Scenario #3: Luís Pedro (driver with reduced mobility) 7
    - Scenario #4: António Videira (Parking Manager) 7
    - Scenario #5: Laura Farias (Maintenance Technician) 7
    - User Story #1: Real-time Parking Availability (High Priority) 8
    - User Story #4: Total of expenses spent on the parking lot (Low Priority) 8
    - User Story #5: Report an unauthorized use of a parking spot (Low Priority) 8
    - User Story #6: Real-time Charger Status (High Priority) 9
    - User Story #7: Combined Parking + Charging Fee Overview (Low Priority) 9
    - User Story #9: Reliability & Physical Compliance (High Priority) 9
    - User Story #10: Real-Time Occupancy & Financial Performance (Low Priority) 9
    - User Story #11: Tariff Management & Issue Oversight (Low Priority) 10
    - User Story #12: Technical Specificity & Remote Diagnostics (Low Priority) 10
    - User Story #13: Infrastructure Mapping & Status Monitoring (High Priority) 10
    - User Story #14: Repair & Update of sensors (High Priority) 10
3. Architecture notebook 10
    - Non-Functional Requirements and Constraints 10
    - Performance and Responsiveness 11
    - Scalability 11
    - Data Consistency and Integrity 11
    - Availability and Fault Tolerance 11
    - Security and Access Control 11
    - Deployment and Portability 12
    - Functional Requirements and Constraints 12
    - Real-Time Monitoring and Alerts 12
    - Parking Simulation 12
    - Reservation Management 12
    - Payment Management 12
    - Analytics and Dashboards 13
    - Notifications 13
    - Architectural view 13
    - Data Sources 13
    - Message Queues 13

Backend..... 13
Frontend..... 14
Base Architecture..... 14
Sequence diagrams..... 16

1. Summary

EasySpot is a real-time parking analytics platform designed to reduce the inefficiency of finding available parking in urban environments. By integrating IoT sensor data with a responsive web interface, the platform delivers live occupancy updates across standard, EV charging, and accessible parking bays. The system serves three user groups: drivers can view live availability, fixed hourly rates, and reserve spots up to 30 minutes in advance; parking operators monitor occupancy, revenue, and tariffs through a dedicated dashboard; and maintenance technicians access remote diagnostics, sensor status maps, and fault alerts to minimize downtime.
Keywords: Real-time parking, IoT sensors, occupancy monitoring, EV charging, accessible parking, urban mobility, parking reservation, sensor diagnostics, operator dashboard.

2. Product concept

Vision Statement

Concept

EasySpot, is a smart parking management platform that provides real-time location and monitoring of parking spaces. It offers a platform across web and mobile interfaces for all users: a driver experience to find, filter, and navigate to available spots, and a data dashboard for operators. The system integrates IoT sensor APIs to capture occupancy data, generate usage statistics, and provide predictive insights.

Inspiration

While there are existing navigation tools like Google Maps or Waze that offer basic location services, they often lack granular, real-time data regarding specific parking availability and specialized needs (such as EV charging status or accessibility features). We were inspired by the growing frustration of "circling the block" in urban centers, which wastes time. Unlike traditional systems, EasySpot differentiates itself by offering routing for electric vehicles and disabled drivers, alongside a management API that allows parking owners to monitor financial returns and hardware health in one place.

Motivation

The motivation behind EasySpot comes from the need to optimize urban space and improve the daily lives of diverse user groups. We wanted to solve the "last-mile" anxiety, the uncertainty of whether a spot will be available upon arrival. By providing real-time data on charger compatibility for EV owners and guaranteed accessibility information for users with reduced mobility, we aim to make city navigation more inclusive. Additionally, we are driven to give parking managers better tools to monitor their infrastructure, moving away from outdated, non-data-driven management methods.

Use Case Diagram

The Use Case Diagram (UCD) illustrating the system's actors and their interactions is presented in Anexos (UCD.png).

Personas

Persona #1: The regular driver

Name: Filipe Teixeira
Age: 25
Location: Coimbra, Portugal
Occupation: Economist
Description: Filipe is a young economist from Coimbra who has to frequently drive to Lisbon for weekly meetings with clients and partners. For Filipe, time is his most valuable resource, and he views parking not just as a necessity but as a logistical challenge that needs to be optimized to ensure his punctuality and professional image. If he feels that he might not be able to get a spot on time, he can make a reservation within 30 minutes before arriving at the parking lot.

Motivation: Wants to minimize search time. He needs to check parking availability in advance to avoid driving in circles, ensuring he arrives at his meetings exactly on time without the stress of being late.

- Appreciates price-to-distance ratios: As an economist, he naturally compares costs. He wants to find the best balance between a low-cost spot and its proximity to his specific meeting location.
- Needs transparent and real-time data: He is frustrated by confusing rate structures and outdated information. He expects the app to show the exact price and a live status of the spots to make quick, data-driven decisions.

Persona #2 : The EV (Electric Vehicle) Driver

Name: Maria Silva
Age: 30
Location: Torres Vedras, Portugal
Occupation: Veterinarian
Description: Maria is a veterinarian living in Torres Vedras who commutes daily to her clinic in the city center. Driving a Tesla, her commute isn't just about the 30-minute journey, but also about ensuring her vehicle remains functional for her busy schedule. For Maria, a parking spot must be more than just a space; it must be a reliable service that supports her commitment to electric mobility while fitting into her demanding professional routine.

Motivation:

- Seeks guaranteed charging infrastructure: Maria's primary motivation is to eliminate her anxiety by finding parking spots equipped with functional EV chargers. She needs to ensure her car is charging while she works so it is ready for her return trip.
- Values punctuality and proximity: As a veterinarian with a tight schedule, she is motivated to find sports close to her office. This avoids long walks and ensures she arrives on time for her job, including during peak hours.
- Demands technical transparency: She is frustrated by the lack of specific information regarding charger types, charging speed, prices, and real-time availability. She expects the app to provide live status updates on chargers to avoid arriving at a station only to find it occupied or out of service, allowing her to make efficient, stress-free decisions.

Persona #3 : The Mobility-Impaired Driver

Name: Luís Pedro
Age: 70
Location: Faro, Portugal
Occupation: Retired

Description: Luís has reduced mobility and uses a wheelchair. He frequently visits clinics and hospitals and needs to ensure that his parking spot is not only available but also practical for his specific physical needs.
Motivation: To maintain independence by finding parking spots that are close to his destination (medical services) and that provide enough space to maneuver his wheelchair safely.

- Appreciates: Accurate real-time information about accessible spots, clear walking (or rolling) distances to entrances, and an easy-to-use interface with high contrast.
- Needs: A way to filter specifically for "Accessible/Disabled" spots; To see the exact distance between the spot and the building entrance; Information on whether spots are being monitored to prevent unauthorized use; Assurance that the spot has the necessary lateral space for a wheelchair ramp or exit.

Persona #4 : The Parking Manager

Name: António Videira
Age: 50
Location: Leiria, Portugal
Occupation: Parking Manager

Description: António manages several parking sites, including the one integrated with EasySpot. He is responsible for day-to-day operations, revenue tracking, and ensuring good customer experience across all lots.
Motivation: Keep occupancy high and predictable while maximising revenue and minimising operational issues. António needs clear, timely information to justify decisions and report performance to stakeholders.

- Appreciates: Actionable key performance indicators (daily entries, occupancy rate, turnover), clear revenue figures and exportable reports, and fast alerts for issues that impact availability or customer satisfaction.
- Needs: A read-only dashboard showing daily entries, occupancy by zone and revenue summaries; visibility of fixed tariffs and historical billing for reconciliation; and a consolidated feed of customer reports and sensor alerts with full details to prioritise actions.

Persona #5 : The Maintenance Technician

Name: Laura Farias
Age: 29
Location: Alvalade, Portugal
Occupation: Maintenance Technician

Description: Laura Farias installs and maintains sensors and gateways for parking sites and expects full access to reported issues, sensor metadata and operational logs so she can diagnose faults and plan interventions.
Motivation: Laura wants complete, timely and precise information about sensor faults and customer reports so she can identify root causes remotely, prioritise repairs efficiently and minimise repeat visits and downtime.

- Appreciates: actionable key performance indicators (sensor uptime, false-positive rate, mean time to repair), detailed diagnostic logs and exportable reports, and immediate contextual alerts (sensor ID,last reading).

- Needs: a consolidated problems feed with sensor metadata; remote diagnostic tools (sensor's status); and the ability to create/assign prioritized maintenance orders with technical notes.

Scenarios

Scenario #1: Filipe Teixeira (regular driver)

Filipe is on his way to his weekly meeting in Lisbon with his clients. Unfortunately, due to unexpected traffic on the bridge, he is running late. He is worried that the time spent searching for a parking spot will further delay his meeting and damage his professional image. During the drive, he opens the EasySpot app to check the number of free spots and current rates at the parking lot, so he knows exactly what to look for when he arrives and how much it will cost him. If he feels that he might not be able to get a spot on time he can make a reservation within 30 minutes before arriving at the parking lot.

Scenario #2: Maria Silva (EV driver)

It is a Tuesday morning, and Maria is preparing for her 30-minute commute from Torres Vedras to her veterinary clinic. Her Tesla's battery is lower than usual, and she knows that finding a spot with a working charger near her office is crucial to avoid stress during her return trip.
Before leaving home, Maria opens the EasySpot app. The app shows her a real-time map with a spot that includes a fast-charger compatible with her vehicle. She can see the current charging rates and the estimated occupancy for the next hour.

Scenario #3: Luís Pedro (driver with reduced mobility)

Luís has a follow-up medical appointment at a clinic. As a wheelchair user, he is often anxious about parking because standard spots do not provide enough space for him to exit his vehicle safely, and accessible spots are frequently occupied by unauthorized drivers.
Before leaving home, Luís uses the EasySpot app to filter for available accessible parking spaces. By checking the app, he can see the exact distance he will need to travel in his wheelchair to the entrance and confirm that the spots are being monitored, giving him the peace of mind that he will be able to attend his appointment on time and without physical strain.

Scenario #4: António Videira (Parking Manager)

António manages a network of parking lots and feels constantly frustrated by the lack of concrete data regarding his operations. The systems he currently uses are limited, offering superficial statistics that do not allow him to understand the true financial performance or the actual flow of customers in his assets.
Before starting his strategy meetings, António uses the EasySpot dashboard to gain a clear view of the business. By accessing the platform, he can monitor the exact number of customers who entered the lot daily and cross-reference that data with financial returns in real-time. Additionally, he validates the current tariffs applied and stays informed about all issues, ensuring that the operation runs flawlessly and that the investment is being profitable.

Scenario #5: Laura Farias (Maintenance Technician)

Laura is responsible for ensuring that the entire technological infrastructure of the park works perfectly. In her daily routine, she frequently faces the problem of receiving breakdown alerts that are too generic, which forces her to travel to the site without the right tools or without knowing exactly which sensor or device is failing.
Through EasySpot, Laura has access to a detailed technical panel where all reported problems appear with their

specificities given by the users. She can consult the status of each sensor and the error history before even leaving the office.

Product requirements (User stories)

User Story #1: Real-time Parking Availability (High Priority)

As a regular driver,
I want to check the number of available parking spots in advance, so that I can minimize my search time and ensure I arrive at my meetings on time.
Given I am using the EasySpot app.
When I open the parking map
Then I should see the number of available spots updated in real time

User Story #2: Price Transparency & Planning (Low Priority)

As a regular driver,
I want to view the parking lot's rate structures and occupancy statistics, so that I can choose the most cost-effective timing for my stay and manage my expenses.
Given I am using the EasySpot app
When I open the pricing and occupancy section
Then I should see the hourly rates, tariffs, and historical occupancy trends

User Story #3: Parking Spot Reservation (Medium Priority)

As a regular driver,
I want to be able to make a reservation within the time limit (30 minutes before arriving)
Given I am driving to the parking lot
When I open the app on the number of available spots
Then I should be able to reserve a spot
And I should see how much it costs

User Story #4: Total of expenses spent on the parking lot (Low Priority)

As a regular driver,
I want to be able to see how much I have spent on the parking lot
Given I am a driver who parks at the parking lot
When I open the app on my expenses
Then I should see the total of my expenses

User Story #5: Report an unauthorized use of a parking spot (Low Priority)

As a client,
I want to be able to fill in a report if I see an unauthorized parking.
Given I am at a parking lot
When I select the option to report an issue
Then I should be able to submit a report

User Story #6: Real-time Charger Status (High Priority)

As an EV driver,
I want to see the real-time status and specification of EV chargers (speed, connector type), so that I can ensure my Tesla is compatible and a spot is available before I arrive.
Given I am searching for EV charging spots
When I open the EasySpot app and filter by EV spots
Then I should see its connector type, charging speed, and current status

User Story #7: Combined Parking + Charging Fee Overview (Low Priority)

As an EV driver,
I want to view combined parking and charging fees so that I can manage my daily commuting expenses and pay everything in a single transaction.
Given I am viewing an EV-enabled parking spot
When I open the pricing details
Then I should see the total estimated cost (parking + charging)

User Story #8: Accessible Spot Filtering & Proximity (High Priority)

As Mobility-Impaired Driver,
I want to filter the parking spots for the specifically designated accessible and see the exact distance to the parking entrance, so that I can ensure the walk the least distance to my medical appointment is manageable and within my physical limits.
Given I am using the app
When I apply the “Accessible Spots” filter
Then I should only see designated accessible spots
And I should see the distance to the parking entrance

User Story #9: Reliability & Physical Compliance (High Priority)

As a Mobility-Impaired driver,
I want to receive real-time confirmation of spot availability and space dimensions, so that I don't risk arriving at a spot that is either illegally occupied by an unauthorized vehicle or too narrow for me to safely deploy my wheelchair.
Given I am viewing an accessible spot
When I open its details
Then I should see its width, lateral clearance, and real-time occupancy

User Story #10: Real-Time Occupancy & Financial Performance (Low Priority)

As a parking operations manager,
I want to access a dashboard that shows the daily number of customers and real-time financial returns, so that I can evaluate the park’s profitability and make data-driven decisions to optimize my investment.
Given I am logged into the operator dashboard
When I access the analytics section
Then I should see daily entries, occupancy rate, and revenue in real time

User Story #11: Tariff Management & Issue Oversight (Low Priority)

As a parking operations manager,
I want to view the current tariff structures and a centralized log of all problems, so that I can ensure the pricing strategy is correct and maintain a high standard of service quality across my facilities.
Given I am in the management dashboard
When I open the tariffs section
Then I should see all current tariffs and their effective dates
And when I open the issues log
Then I should see all reported problems with timestamps and categories

User Story #12: Technical Specificity & Remote Diagnostics (Low Priority)

As a field maintenance technician,
Accessing sensor logs and connection data will allow me to troubleshoot issues remotely and be fully prepared before traveling to the site.
Given I am viewing the maintenance panel
When I select a reported issue
Then I should see the sensor ID
And I should see the full error history

User Story #13: Infrastructure Mapping & Status Monitoring (High Priority)

As a field maintenance technician,
I want to see a live map of all hardware components and their individual status, so that I can perform proactive maintenance and ensure the parking ecosystem remains fully operational for all users.
Given I am in the technical dashboard
When I open the infrastructure map
Then I should see all sensors and their information
And each component should display its real-time operational status

User Story #14: Repair & Update of sensors (High Priority)

As a field maintenance technician,
I want to be able to update the state of the sensors when I repair a malfunction. So that the system accurately reflects their current operational condition.
Given I am on the technical dashboard
When I repair a sensor malfunction
Then I should be able to update the sensor's status
And the error statistics should be updated accordingly to reflect the change.

3. Architecture notebook

Key quality requirements

Non-Functional Requirements and Constraints

As a real-time parking management and analytics platform, EasySpot must provide low-latency data flow, reliable event processing, secure access control, and persistent storage for both operational and historical data.

The system supports live parking maps, reservations, payments, dashboards, alerts, and analytics for different user roles, including drivers, managers, and technicians.

Performance and Responsiveness

EasySpot must process real-time parking events with low latency.
When a parking spot changes state, for example, from occupied to free, the update should be reflected on the frontend through WebSockets in near real time, preferably within 2 seconds under normal operating conditions.
REST operations such as searching parks, creating reservations, updating profiles, or viewing dashboards should remain responsive and provide a smooth user experience.

Scalability

The system must continue operating correctly as the number of users, parking lots, sensors, and events increases.
Kafka is used to decouple event producers from backend processing, allowing the platform to handle many simultaneous sensor, OCR, and gate events asynchronously.
The application should support hundreds of concurrent sensor updates and approximately 500 to 1000 simultaneous users without noticeable degradation in normal usage scenarios.

Data Consistency and Integrity

EasySpot must maintain accurate and consistent parking data.
The backend validates and normalises incoming sensor events before updating parking spot states. It must correctly distinguish between regular, EV, accessible, reserved, occupied, free, and out-of-service spots to avoid misleading users.
Reservation and payment data must remain consistent, especially when availability changes, reservations are updated, or Stripe payment operations fail. Historical data stored in TimescaleDB must also remain reliable for analytics and reporting.

Availability and Fault Tolerance

The system should remain resilient when individual components experience temporary failures.
Kafka helps prevent data-processing bottlenecks by buffering events between simulators and the backend. Stripe-related failures should not unnecessarily invalidate confirmed reservations, and sensor faults should be detected and reported to technicians.
The platform should also degrade gracefully when optional external services, such as email or payment services, are unavailable.

Security and Access Control

EasySpot must protect user data and restrict access according to user roles.
Authentication is handled through Authentik, while the backend validates JWT tokens on incoming requests. Role-based access control ensures that drivers, managers, and technicians can only access the endpoints and dashboards appropriate to their permissions.
Sensitive operations, such as payment handling, profile management, reservation management, and technician actions, must require authenticated access.

Deployment and Portability

The system is constrained by the requirement to be fully containerized using Docker and Docker Compose.
This allows the frontend, backend, PostgreSQL, TimescaleDB, Kafka, Authentik, Nginx, sensor simulators, and supporting services to be deployed consistently across development and server environments.
Nginx acts as the reverse proxy, centralising access to the frontend, backend API, and authentication service.

Functional Requirements and Constraints

The main functionalities of EasySpot are focused on real-time parking availability, reservations, payments, sensor simulation, alerts, and operational analytics.
The platform must support different user roles and expose role-specific features for drivers, managers, and technicians.

Real-Time Monitoring and Alerts

EasySpot must display live parking availability for regular, EV, and accessible spots.
The frontend receives real-time updates through WebSockets whenever parking spot states, reservations, alerts, or sensor conditions change.
The system must notify users and staff about relevant events, such as parking availability changes, reservation updates, payment status, sensor faults, and maintenance-related alerts.

Parking Simulation

The simulation environment must generate realistic parking events without requiring physical sensors.
Python agents simulate IR sensors, OCR camera readings, gate interactions, sensor faults, and recovery events. These events are published to Kafka and consumed by the backend as if they came from real parking infrastructure.
This allows the team to test the event pipeline, real-time map updates, reservation flow, technician dashboard, and analytics features.

Reservation Management

Drivers must be able to search for parking lots, check availability, create reservations, update reservations, and cancel reservations.
Before confirming or changing a reservation, the backend must verify availability, calculate the expected cost, and apply the appropriate lifecycle rules.
Reservation changes should support cost previews before final confirmation.

Payment Management

EasySpot must support payment processing through Stripe.
The system must create payment intents, support saved payment methods, process payment adjustments, handle refunds, and receive Stripe webhook events.
Payment records must be stored for traceability, while parking sessions and revenue-related data must be available for reporting and analytics.

Analytics and Dashboards

Managers must be able to access operational dashboards with metrics such as daily entries, occupancy rates, revenue summaries, parking trends, tariff information, and historical usage.
Technicians must be able to view assigned parks, sensor status, recent alerts, reported issues, and maintenance-related information.

Notifications

The system must send notifications through real-time WebSocket channels and email.
WebSockets are used for live alerts and dashboard updates, while email is used for reservation confirmations, payment information, reservation changes, cancellations, and important operational alerts.

Architectural view

To make the EasySpot architecture easier to understand, the system can be divided into four main parts: Frontend, Backend, Data Sources, and Message Queues. Together, these layers support real-time parking availability, reservations, payments, access control, notifications, and operational analytics.

Data Sources

The Data Source Layer is mainly composed of a Python-based simulation environment that represents a network of virtual parking sensors and gate devices.
These agents generate real-time events that mimic physical parking infrastructure, including parking spot occupancy changes, IR sensor status updates, sensor faults and recoveries, OCR licence plate readings, and gate-related events.
The simulators obtain parking context from the backend, such as available parking lots and parking spots, and then publish events that reflect the current simulated state of the system.

Message Queues

The Message Queue Layer uses Apache Kafka as the asynchronous event broker between the simulators and the backend.
Kafka decouples event generation from event processing, allowing sensor, OCR, and gate events to be produced independently from backend consumption. This improves scalability, resilience, and fault tolerance, especially when many parking spots or devices are producing updates at the same time.
Kafka is also used for internal event flows, such as occupancy changes, reservation events, alert events, and gate command responses.

Backend

The Backend Layer is developed with Spring Boot and acts as the core of the EasySpot platform. It is responsible for event processing, business rules, authentication enforcement, reservations, billing, notifications, analytics, and integration with external services.
The backend consumes Kafka events from the virtual sensors, validates and normalises them, updates parking spot states, records sensor activity, manages OCR plate readings, and coordinates gate operations.
It uses PostgreSQL for relational and transactional data, such as users, vehicles, parking lots, parking spots, reservations, tariffs, payment records, and profile information. TimescaleDB is used for time-series and historical data, such as occupancy snapshots, parking sessions, alerts, revenue trends, and operational metrics.

This layer also integrates with external services, including Stripe for payment processing and refunds, Authentik for authentication and JWT-based access control, and SMTP email services for automated notifications such as booking confirmations, payment updates, cancellations, and operational alerts.

Frontend

The Frontend Layer is implemented as a React web application powered by Vite.
It provides different interfaces according to the user role. Drivers can search for parking lots, manage vehicles, create reservations, and handle payments. Managers can monitor occupancy, revenue, tariffs, and operational performance. Technicians can consult assigned parks, sensor health, faults, and alerts.
The frontend communicates with the backend via REST APIs for standard operations such as authentication-aware requests, search, booking, reservation updates, payments, and dashboard queries.
It also maintains WebSocket connections for real-time updates, including live parking map changes, reservation updates, sensor faults, technician alerts, and dashboard notifications. This ensures that users receive up-to-date system information without having to manually refresh the page.
All external access is routed through Nginx, which acts as the reverse proxy for the frontend, backend API, and Authentik authentication service.

Base Architecture

The base architecture of the system, including component interactions and deployment structure, is presented in Anexos (BaseArchiteture.png).

Layered Architecture

The layered architecture, depicting the separation of concerns across presentation, business logic, and data layers, is presented in Anexos (LayeredArchitecture.png).

Module interactions

1. **System Overview**

The EasySpot system is composed of several interconnected modules that work together to provide real-time parking management, reservations, payments, monitoring, and analytics.
The architecture combines synchronous REST communication, asynchronous Kafka event processing, WebSocket-based real-time updates, PostgreSQL for transactional data, and TimescaleDB for historical and time-series data.

2. **Virtual Sensors and Kafka**

Virtual sensors are simulated by Python agents that retrieve parking context from the backend and publish events to Kafka topics.
These events include parking spot occupancy changes, IR sensor status updates, OCR plate readings, sensor faults, sensor recovery events, and gate-related events.
Kafka acts as the asynchronous communication layer, decoupling event producers from backend processing and improving scalability and resilience.

3. **Spring Boot Backend and Data Storage**

The Spring Boot backend consumes Kafka events through dedicated listeners for occupancy, OCR, sensors, and gate events.
Incoming events are validated, normalised, and routed to the appropriate domain services. Real-time operational data, such as users, parking lots, parking spots, reservations, payments, vehicles, tariffs, and profiles, is stored in PostgreSQL.

Historical and analytical data, such as occupancy snapshots, parking sessions, revenue records, and alerts, is stored in TimescaleDB, which is optimised for time-series workloads.

4. **Occupancy Module**

The Occupancy Module processes parking spot events received from Kafka.
It validates the spot identifier, normalises the received status, checks whether the transition is valid, and updates the corresponding parking spot state in the database. When a valid change occurs, the module persists the new status, records sensor activity, captures occupancy snapshots when required, and publishes internal occupancy events.
These updates are also used to keep the parking map and dashboards synchronised in real time.

5. **OCR and Gate Modules**

The OCR Module processes licence plate readings generated at parking lot entrances and exits. These readings can be associated with vehicles, reservations, and parking sessions.
The Gate Module manages gate commands and responses through Kafka. It coordinates gate opening logic with reservation status, vehicle identification, and payment/session information, ensuring that access control is consistent with the system state.

6. **Booking Module**

The Booking Module handles reservation requests received through REST Controllers.
Before confirming a reservation, it validates the authenticated user, checks real-time parking availability, calculates the expected cost, and applies the reservation lifecycle rules. Confirmed reservations are stored in the ReservationRepository.
The module also supports reservation updates and cancellations. When a reservation is changed, the system can preview the new cost before persisting the update or contacting Stripe. Reservation events are also pushed to the frontend through WebSockets.

7. **Billing Module**

The Billing Module integrates with Stripe to manage payments.
It creates PaymentIntents for reservations, stores payment records, supports saved payment methods, processes payment adjustments, handles refunds, and receives Stripe webhook events.
Payment records are stored in PostgreSQL, while parking sessions and revenue-related historical data are stored in TimescaleDB. This separation allows transactional payment data and analytical parking-session data to be handled efficiently.

8. **Notification Module**

The Notification Module centralises operational and user-facing notifications.
It receives events from modules such as Occupancy, Booking, Billing, Sensor, and Gate. It then builds alert payloads and delivers them through real-time WebSocket channels and email.
WebSockets are used for live dashboard updates, parking availability changes, reservation updates, and technician alerts. Email notifications are used for reservation confirmations, payment information, reservation updates, cancellations, and important operational alerts.
Notification and alert history is persisted so managers and technicians can consult previous events.

9. **Analytics Module**

The Analytics Module aggregates data from occupancy snapshots, parking sessions, billing records, sensor status, and alerts.

It exposes operational reports through REST Controllers for manager and technician dashboards. These reports include daily entries, occupancy rates, revenue summaries, parking usage trends, sensor health, recent alerts, and tariff-related information.
TimescaleDB is used as the main storage layer for historical metrics and time-series analytics.

10. Auth Module

The Auth Module manages authentication and access control using Authentik and JWT tokens.
Authentik handles login flows, session management, identity data, and role assignment. The frontend obtains JWT tokens from Authentik and sends them to the backend with each request.
The backend validates these tokens and enforces role-based permissions for drivers, managers, and technicians. Each role has access to specific endpoints, dashboards, and actions.

11. Frontend, Proxy, and Presentation Layer

The frontend web application consumes REST endpoints and WebSocket streams exposed by the Spring Boot backend.
Drivers can search for parking lots, manage vehicles, create reservations, and handle payments. Managers can monitor occupancy, revenue, tariffs, and operational performance. Technicians can view assigned parks, sensor status, faults, and alerts.
All external traffic is routed through Nginx, which centralises access to the frontend, backend API, and Authentik. This reverse proxy simplifies ingress, routing, and deployment configuration.

Sequence diagrams

The following diagrams describe the behaviour of the EasySpot system across four key scenarios.

User Opens EasySpot (Real-Time Data Load)

This diagram describes the initial data load flow triggered when a user opens the EasySpot application.

- On startup, the Frontend (Vite) connects to the Backend via WebSocket (STOMP over SockJS), authenticating with a Bearer token. It then sends an HTTP request to fetch the current parking occupancy state from the Backend REST API.
- The Backend queries the PostgreSQL/TimescaleDB database and returns the current spot availability to the Frontend, which renders the parking map with live occupancy data.
- Meanwhile, the Sensors simulator continuously publishes occupancy events (spot state transitions: free, occupied, reserved, out_of_service) to Kafka on the parking-spot-events topic. The Backend Kafka listener consumes these events, resolves the new spot state, persists the update to the database, and after the transaction commits, broadcasts an OccupancyEvent to all WebSocket subscribers on /topic/occupancy/parks/{parkId}.
- The Frontend receives these WebSocket push events and updates the parking map in real time without polling.

The flow of real-time data loading when a user opens EasySpot is detailed in the sequence diagram presented in Anexos (UserOpensEasySpot.png).

Spot Reservation (User Story #3)

This diagram covers the reservation flow triggered by a regular driver such as Filipe, who needs to secure a parking spot up to 30 minutes before arriving.

- The flow starts when the user submits a reservation request through the Frontend, which forwards it to the Backend.

- The Backend verifies real-time spot availability and temporarily locks the spot for 30 minutes. At this point the flow branches: if the spot is available, the Backend proceeds to calculate the estimated cost and initiates a payment intent via the external billing integration; if unavailable, an error response is returned to the Frontend and the flow terminates.
- Once confirmed, the booking record is persisted and a confirmation response is returned to the Frontend, which displays the reserved spot and its cost to the user.

The sequence diagram for spot reservation (User Story #3) is presented in Anexos (SpotReservation.png).

Real-Time Occupancy Update (User Story #1)

This diagram describes when a sensor detects a change in a parking spot's state and the update is propagated in real time to all connected clients.

- The flow begins when the Sensors simulator publishes a ParkingSpotEvent to Kafka on the parking-spot-events topic. Each event carries the spot identifier, the new status (free, occupied, reserved, or out_of_service), and an optional payload with context such as the reason for a state change.
- The Backend Kafka listener (@KafkaListener) consumes the event and first validates that the required fields are present. It then fetches the current spot record from PostgreSQL and delegates the transition decision to the SpotStateResolver, which enforces valid state machine transitions and rejects duplicates or invalid moves. If the transition is rejected, the event is silently dropped and no database write occurs.
- If the transition is accepted, the Backend updates the spot status in PostgreSQL/TimescaleDB and, if enough time has elapsed since the last snapshot, captures a new occupancy snapshot for analytics. Both operations run inside a single database transaction. Only after the transaction commits does the Backend broadcast an OccupancyEvent: it publishes the event to the occupancy-events Kafka topic and simultaneously pushes it via WebSocket (STOMP) to all connected Frontend clients subscribed on /topic/occupancy/parks and /topic/occupancy/parks/{parkId}.
- The Frontend receives the push event and updates the parking map in real time without polling, keeping end-to-end latency within the system's 5-second requirement.

The sequence diagram for real-time occupancy update (User Story #1) is presented in Anexos A (RealTimeOccupancyUpdate.png).

EV Charger Status (User Story #6)

This diagram describes the flow triggered when an EV driver such as Maria searches for an available charging spot compatible with her vehicle.

- The flow begins when Maria submits a request through the frontend.
- The request reaches the system, which first authenticates the user via Authentik before proceeding.
- Once authorised, the system queries the database to retrieve live EV charger availability, technical specifications, and applicable parking and charging rates, all handled by the same internal service.
- The database returns the results to the system, which assembles the full response and returns it to the Frontend.
- The frontend then displays a live map showing only available EV spots with their technical specifications and combined cost estimate.

The sequence diagram for EV charger status (User Story #6) is presented in Anexos (EVChargerStatus.png).

Sensor Fault Alert (User Story #13)

This diagram describes the implemented sensor fault and recovery flow for User Story #13, focused on infrastructure mapping and real-time status monitoring.The flow starts when a sensor or simulator publishes a fault or recovery event to Kafka, such as sensor.fault, device.fault, sensor.recovered, or device.recovery. The

backend consumes this event through the SensorEventKafkaListener and forwards it to the SensorLogsService, which updates the corresponding sensor record in the SensorRegistry.

- When a fault is detected, the sensor status is changed to OFFLINE or DEGRADED. The system checks whether there is already an open alert for that sensor. If no open alert exists, it creates a new SENSOR alert with state OPEN and severity CRITICAL. The backend also records the decision in the sensor decision history and publishes a real-time update through WebSocket/STOMP to the assigned technician dashboard.
- Laura can then view the affected sensor in the technical infrastructure map, see its current operational status, and consult the sensor details, logs, alert information, and error history remotely. If she starts working on the issue, the related alert can move from OPEN to IN_PROGRESS.
- After the repair is complete, Laura updates the sensor status from the technician dashboard. If the sensor is marked as OPERATIONAL, the backend updates the sensor registry, resolves the associated open alert by changing it to RESOLVED, stores the resolution timestamp and notes, and publishes another real-time update. The technician dashboard and infrastructure map are then refreshed, showing the component as operational again and keeping the alert history for future diagnostics.

The sequence diagram for sensor fault alert (User Story #13) is presented in Anexos A (SensorFaultAlert.png).

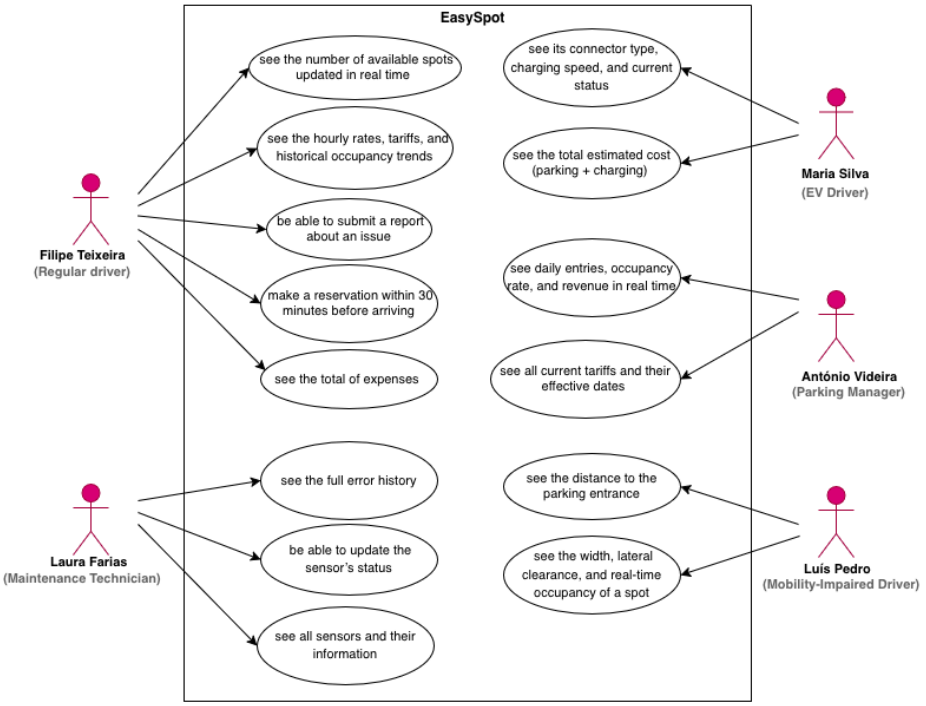
Information Perspective

The Entity-Relationship Diagram (ERD) representing the database schema is presented in Anexos (DER.png).

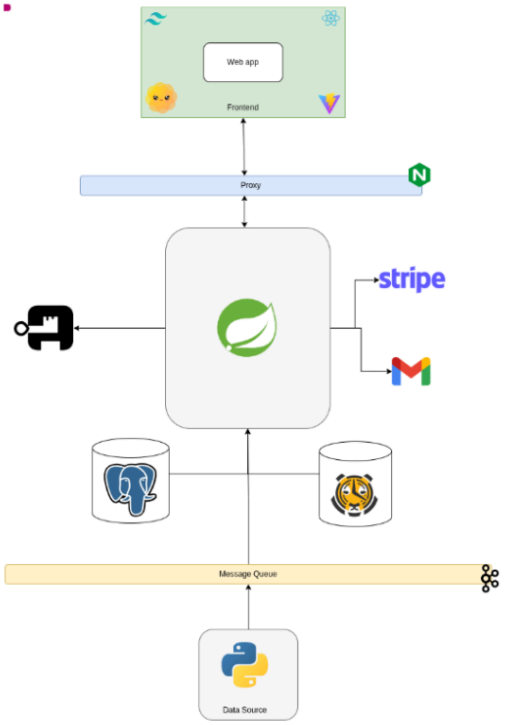
Conclusion

EasySpot successfully addressed the core challenge of urban parking inefficiency by delivering a real-time occupancy platform tailored to four distinct user groups. The architecture built around Kafka, Spring Boot, and WebSockets proved well-suited to meet the system's key non-functional requirements of low latency, scalability, and data consistency. The use of personas, scenarios, and user stories kept development decisions grounded in real user needs, while the sequence diagrams validated the technical feasibility of the most critical flows. Overall, EasySpot demonstrates how IoT integration and real-time data pipelines can make urban mobility more efficient and inclusive.

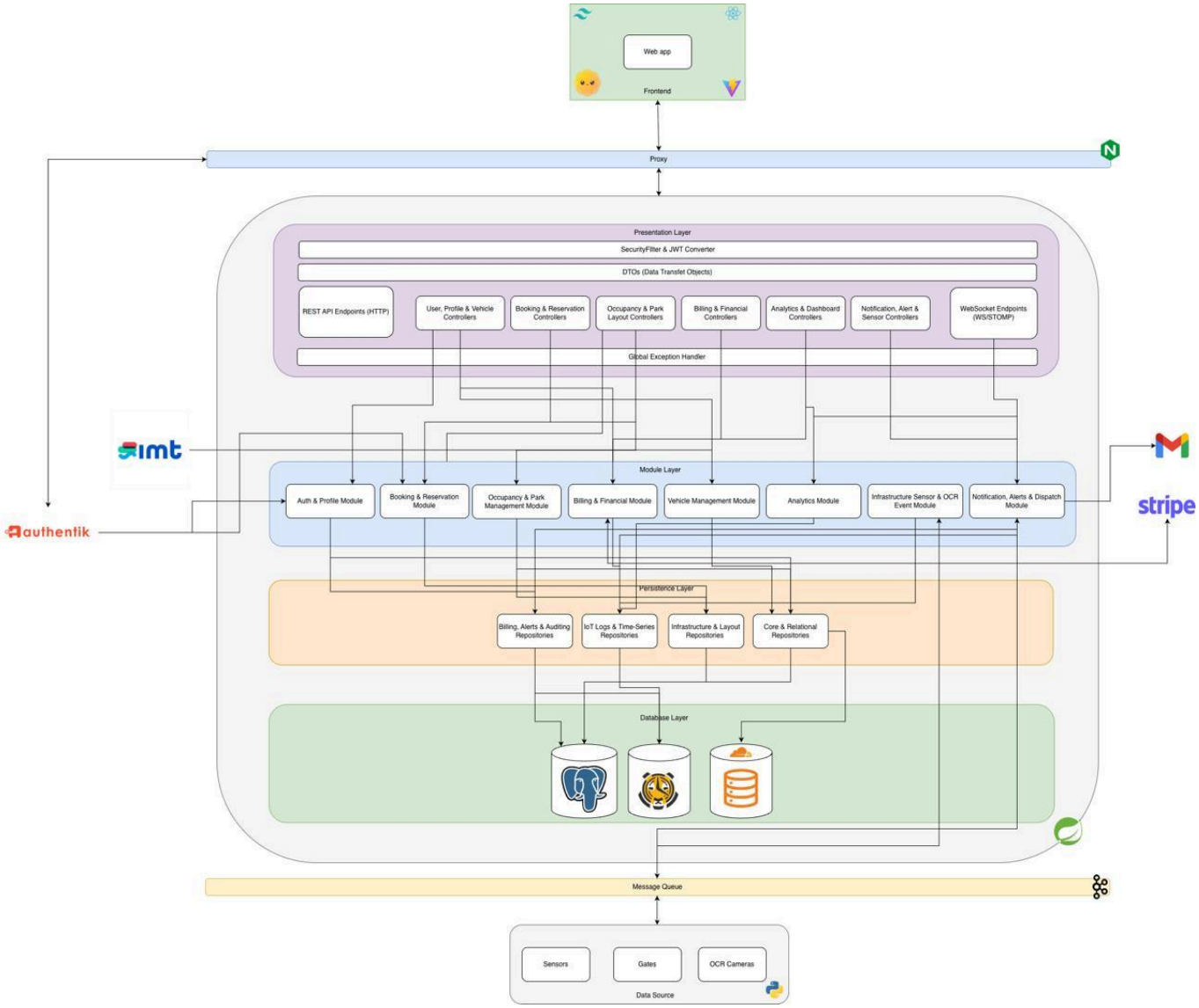
Anexo:



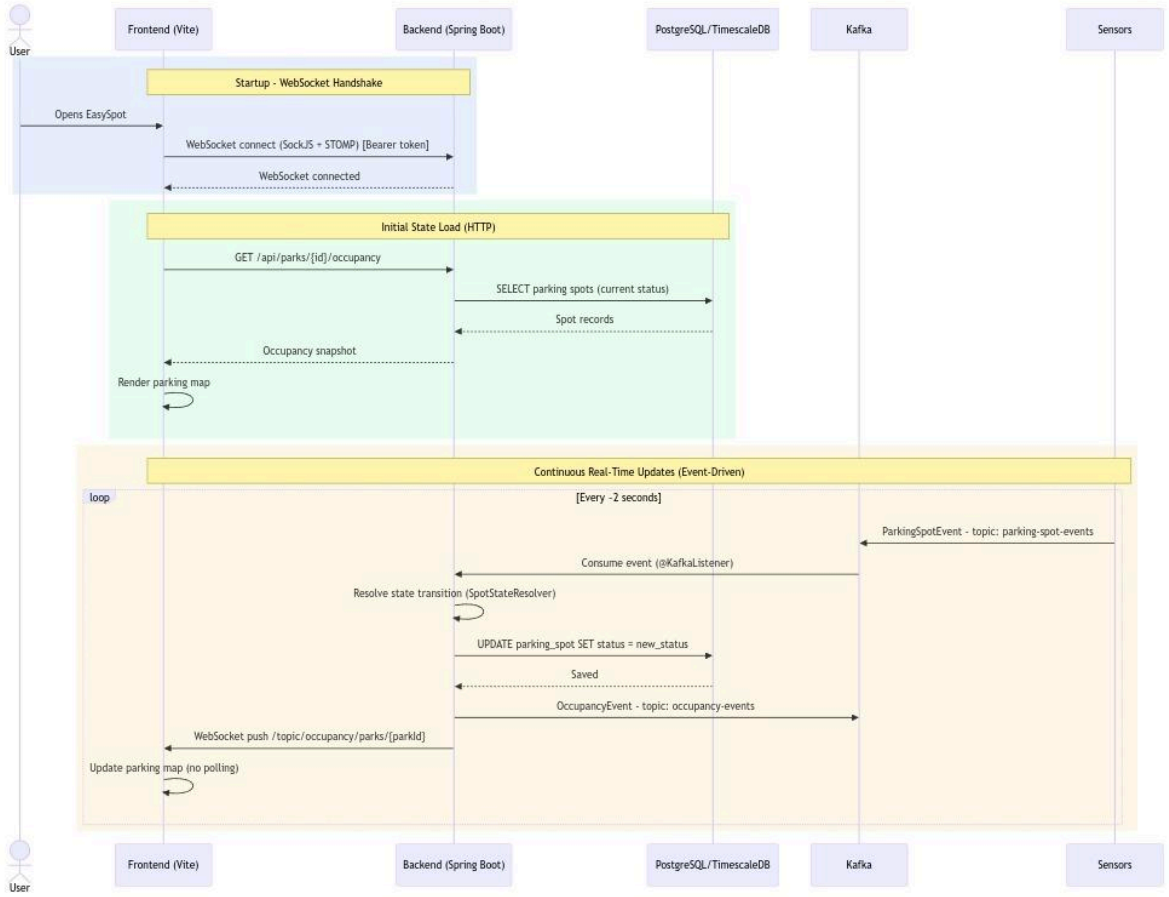
UCD.png



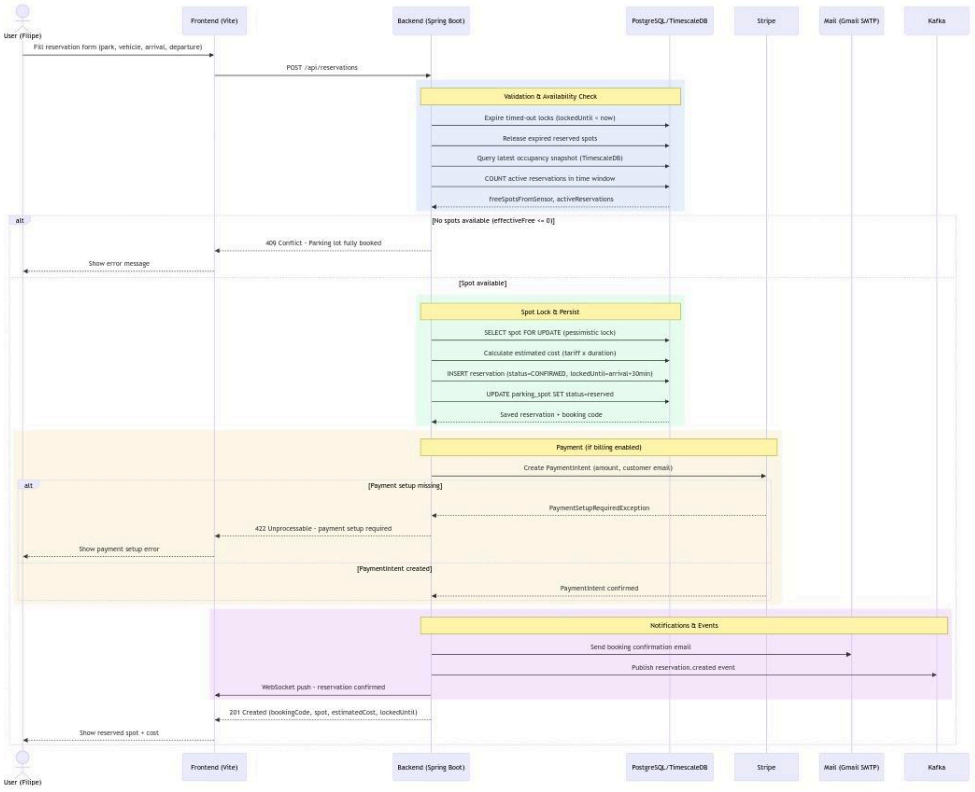
BaseArchitecture.png



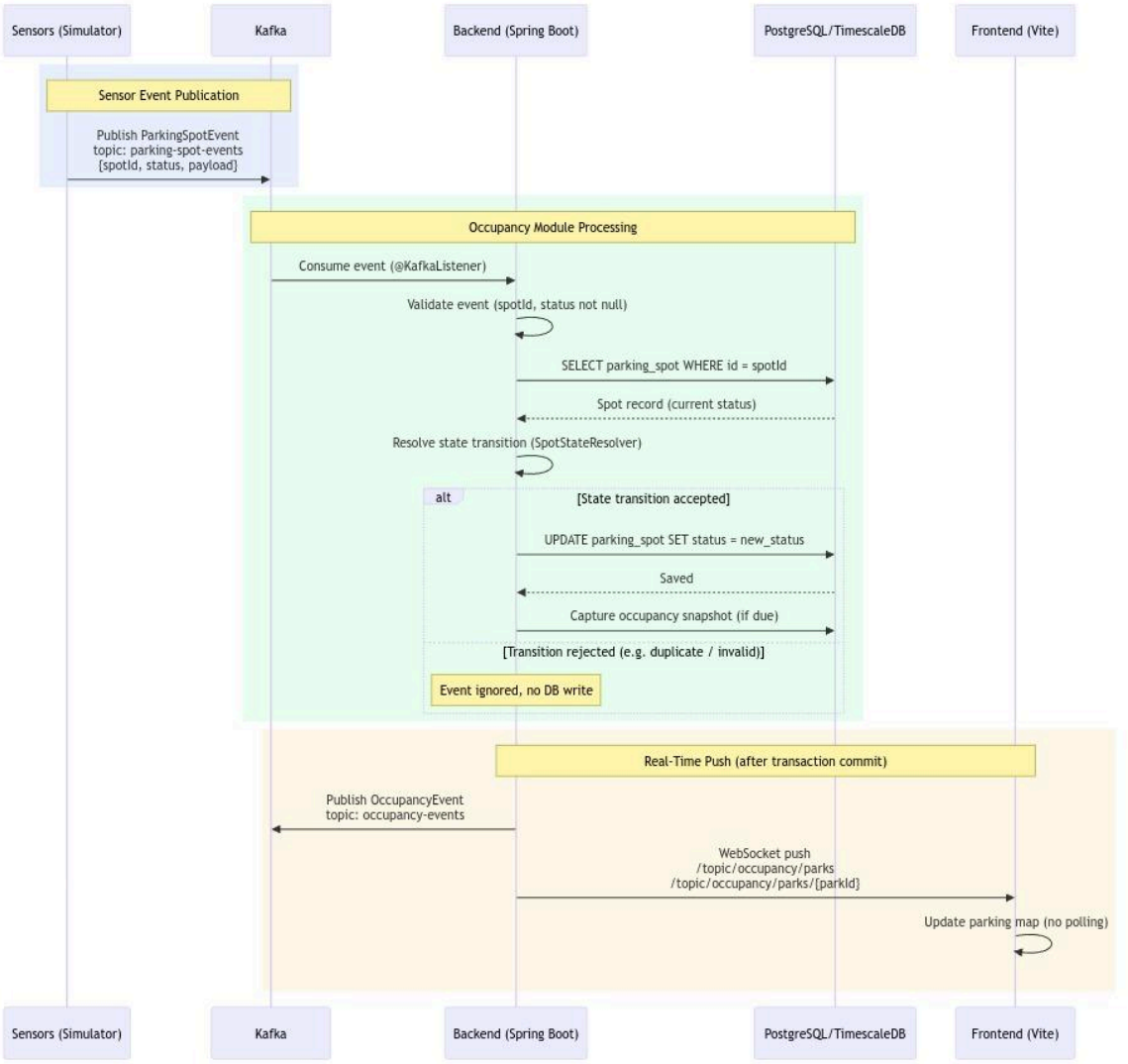
LayeredArchitecture.png



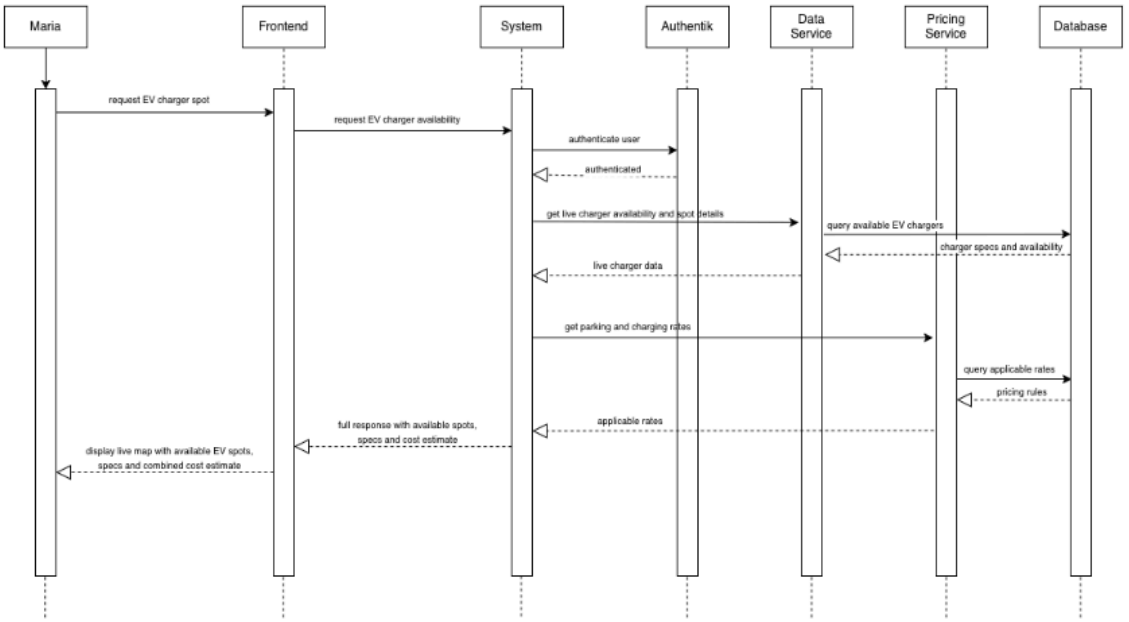
UserOpensEasySpot.png



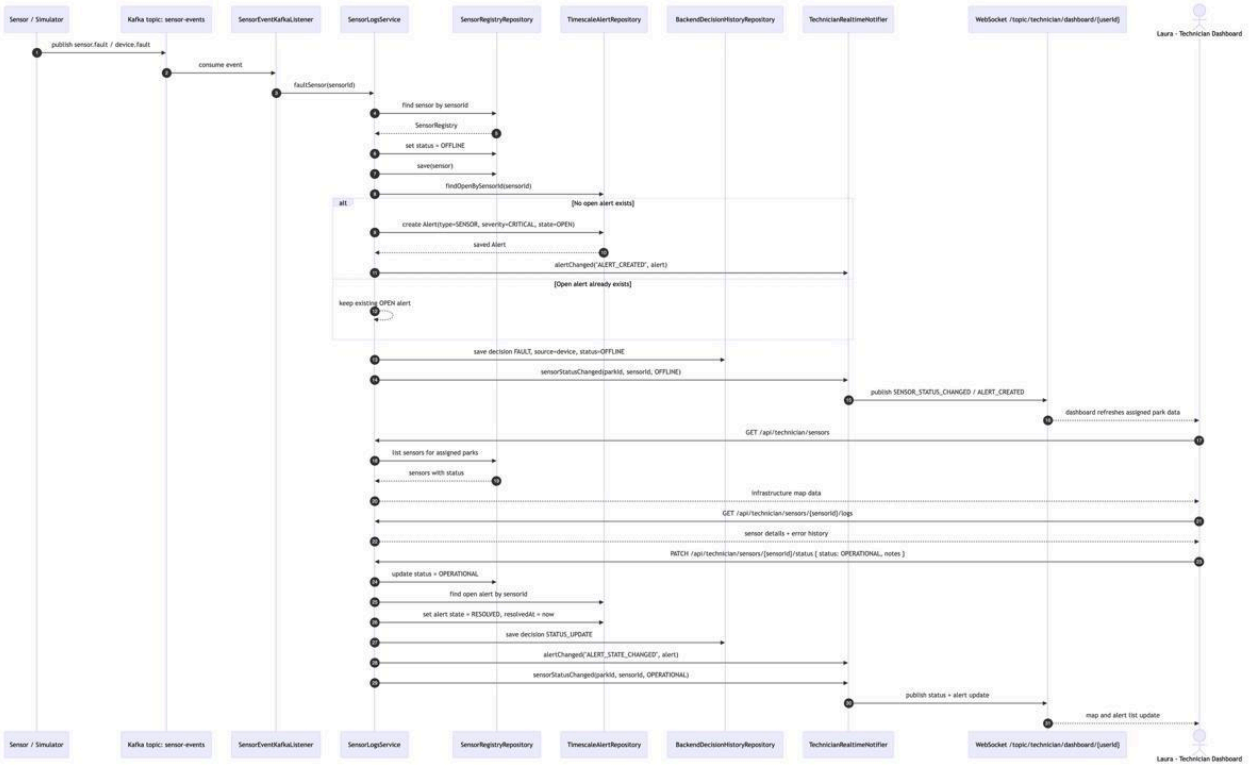
SpotReservation.png



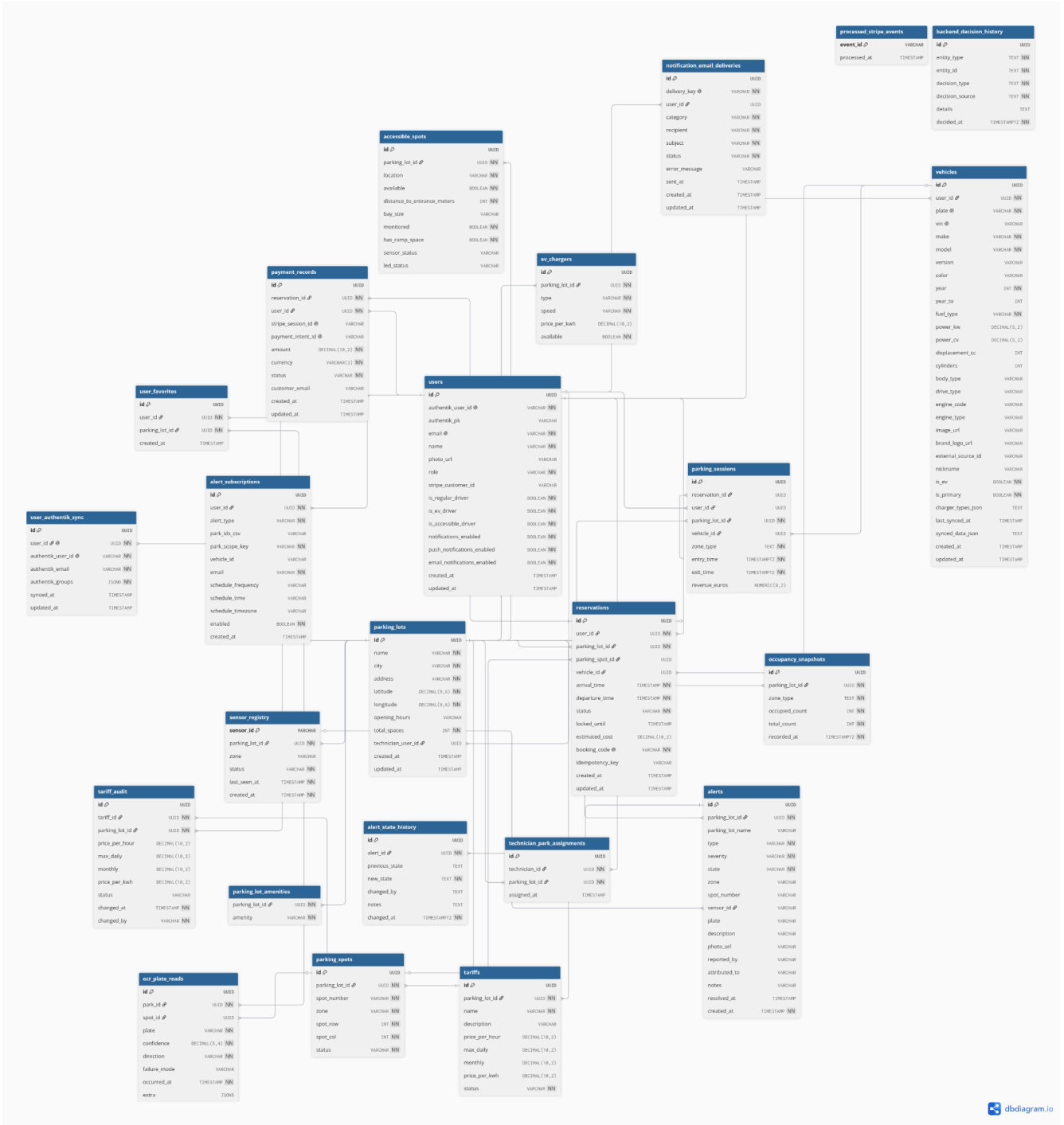
RealTimeOccupancyUpdate.png



EVChargerStatus.png



SensorFault.png


processed_stripe_events
event_id VARCHAR
processed_at TIMESTAMP
backend_decision_history
id UUID
entity_type TEXT NN
entity_id TEXT NN
decision_type TEXT NN
decision_source TEXT NN
details TEXT
decided_at TIMESTAMPTZ NN
notification_email_deliveries
id UUID
delivery_key VARCHAR NN
user_id UUID
category VARCHAR NN
recipient VARCHAR NN
subject VARCHAR NN
status VARCHAR NN
error_message VARCHAR
sent_at TIMESTAMP
created_at TIMESTAMP
updated_at TIMESTAMP
accessible_spots
id UUID
parking_lot_id UUID NN
location VARCHAR NN
available BOOLEAN NN
distance_to_entrance_meters INT NN
bay_size VARCHAR
monitored BOOLEAN NN
has_ramp_space BOOLEAN NN
sensor_status VARCHAR
led_status VARCHAR
vehicles
id UUID
user_id UUID NN
plate VARCHAR NN
vin VARCHAR
make VARCHAR NN
model VARCHAR NN
version VARCHAR
color VARCHAR
year INT NN
year_to INT
fuel_type VARCHAR NN
power_kw DECIMAL(5,2)
power_cv DECIMAL(5,2)
displacement_cc INT
cylinders INT
body_type VARCHAR
drive_type VARCHAR
engine_code VARCHAR
engine_type VARCHAR
image_url VARCHAR
brand_logo_url VARCHAR
external_source_id VARCHAR
nickname VARCHAR
is_ev BOOLEAN NN
is_primary BOOLEAN NN
charger_types_json TEXT
last_synced_at TIMESTAMP
synced_data_json TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
ev_chargers
id UUID
parking_lot_id UUID NN
type VARCHAR NN
speed VARCHAR NN
price_per_kwh DECIMAL(10,2)
available BOOLEAN NN
payment_records
id UUID
reservation_id UUID NN
user_id UUID NN
stripe_session_id VARCHAR
payment_intent_id VARCHAR
amount DECIMAL(10,2) NN
currency VARCHAR(3) NN
status VARCHAR NN
customer_email VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP
users
id UUID
authentik_user_id VARCHAR NN
authentik_pk VARCHAR
email VARCHAR NN
name VARCHAR NN
photo_url VARCHAR
role VARCHAR NN
stripe_customer_id VARCHAR
is_regular_driver BOOLEAN NN
is_ev_driver BOOLEAN NN
is_accessible_driver BOOLEAN NN
notifications_enabled BOOLEAN NN
push_notifications_enabled BOOLEAN NN
email_notifications_enabled BOOLEAN NN
created_at TIMESTAMP
updated_at TIMESTAMP
user_favorites
id UUID
user_id UUID NN
parking_lot_id UUID NN
created_at TIMESTAMP
alert_subscriptions
id UUID
user_id UUID NN
alert_type VARCHAR NN
park_ids_csv VARCHAR
park_scope_key VARCHAR NN
vehicle_id VARCHAR
email VARCHAR NN
schedule_frequency VARCHAR
schedule_time VARCHAR
schedule_timezone VARCHAR
enabled BOOLEAN NN
created_at TIMESTAMP
user_authentik_sync
id UUID
user_id UUID NN
authentik_user_id VARCHAR NN
authentik_email VARCHAR NN
authentik_groups JSONB NN
synced_at TIMESTAMP
updated_at TIMESTAMP
parking_sessions
id UUID
reservation_id UUID
user_id UUID
parking_lot_id UUID NN
vehicle_id UUID
zone_type TEXT NN
entry_time TIMESTAMPTZ NN
exit_time TIMESTAMPTZ NN
revenue_euros NUMERIC(8,2)
reservations
id UUID
user_id UUID NN
parking_lot_id UUID NN
parking_spot_id UUID
vehicle_id UUID
arrival_time TIMESTAMP NN
departure_time TIMESTAMP NN
status VARCHAR NN
locked_until TIMESTAMP
estimated_cost DECIMAL(10,2)
booking_code VARCHAR NN
idempotency_key VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP
parking_lots
id UUID
name VARCHAR NN
city VARCHAR NN
address VARCHAR NN
latitude DECIMAL(9,6) NN
longitude DECIMAL(9,6) NN
opening_hours VARCHAR
total_spaces INT NN
technician_user_id UUID
created_at TIMESTAMP
updated_at TIMESTAMP
occupancy_snapshots
id UUID
parking_lot_id UUID NN
zone_type TEXT NN
occupied_count INT NN
total_count INT NN
recorded_at TIMESTAMPTZ NN
sensor_registry
sensor_id VARCHAR
parking_lot_id UUID NN
zone VARCHAR
status VARCHAR NN
last_seen_at TIMESTAMP NN
created_at TIMESTAMP NN
tariff_audit
id UUID
tariff_id UUID NN
parking_lot_id UUID NN
price_per_hour DECIMAL(10,2)
max_daily DECIMAL(10,2)
monthly DECIMAL(10,2)
price_per_kwh DECIMAL(10,2)
status VARCHAR
changed_at TIMESTAMP NN
changed_by VARCHAR NN
alert_state_history
id UUID
alert_id UUID NN
previous_state TEXT
new_state TEXT NN
changed_by TEXT
notes TEXT
changed_at TIMESTAMPTZ NN
technician_park_assignments
id UUID
technician_id UUID NN
parking_lot_id UUID NN
assigned_at TIMESTAMP
alerts
id UUID
parking_lot_id UUID NN
parking_lot_name VARCHAR
type VARCHAR NN
severity VARCHAR NN
state VARCHAR NN
zona VARCHAR
spot_number VARCHAR
sensor_id VARCHAR
plate VARCHAR
description VARCHAR
photo_url VARCHAR
reported_by VARCHAR
attributed_to VARCHAR
notes VARCHAR
resolved_at TIMESTAMP
created_at TIMESTAMP NN
parking_lot_amenities
parking_lot_id UUID NN
amenity VARCHAR NN
parking_spots
id UUID
parking_lot_id UUID NN
spot_number VARCHAR NN
zone VARCHAR NN
spot_row INT NN
spot_col INT NN
status VARCHAR NN
tariffs
id UUID
parking_lot_id UUID NN
name VARCHAR NN
description VARCHAR
price_per_hour DECIMAL(10,2)
max_daily DECIMAL(10,2)
monthly DECIMAL(10,2)
price_per_kwh DECIMAL(10,2)
status VARCHAR NN
ocr_plate_reads
id UUID
park_id UUID NN
spot_id UUID
plate VARCHAR NN
confidence DECIMAL(5,4) NN
direction VARCHAR NN
failure_mode VARCHAR
occurred_at TIMESTAMP NN
extra JSONB
dbdiagram.io

DER.png